

# RLPlace: Using Reinforcement Learning and Smart Perturbations to Optimize FPGA Placement

Mohamed A. Elgammal<sup>1b</sup>, Kevin E. Murray<sup>2b</sup>, *Member, IEEE*, and Vaughn Betz<sup>3b</sup>, *Fellow, IEEE*

**Abstract**—Simulated annealing (SA) is one of the most common FPGA placement techniques, and is used both as a standalone algorithm and to improve an initial analytical placement. While SA-based placers can achieve high-quality results, they suffer from long runtimes. In this article, we introduce *RLPlace*, a novel SA-based FPGA placer that utilizes both reinforcement learning (RL) and targeted perturbations (directed moves). The proposed moves target both wirelength and timing optimization and explore the solution space more efficiently than traditional random moves while preventing oscillation in the Quality of Results (QoR). RL techniques are used to dynamically select the most effective move types as optimization progresses. The experimental results show that *RLPlace* outperforms the widely used VTR 8 placer across all runtime/quality tradeoff points, achieving better QoR placement solutions in less runtime. On average, across the Titan23 suite of large FPGA benchmarks, *RLPlace* can reduce CPU time by 2.5× with result quality comparable to VTR 8, or improve wirelength by 8% (at a high CPU time budget) –26% (at a low CPU time budget) versus VTR 8.0 given the same CPU time.

**Index Terms**—Directed moves, FPGA, placement, reinforcement learning (RL).

## I. INTRODUCTION

**B**OTH the capacity and the architectural complexity of FPGAs have been continuously increasing in accordance with Moore's law, putting pressure on the runtime of FPGA computer-aided design (CAD) algorithms [1]. Recently, Xilinx announced the Virtex UltraScale+ VU19P FPGA, which contains 9 million logic cells and almost 35 billion transistors [2] while Intel has announced the Stratix 10 GX 10M device with more than 10.2 million logic cells and over 43 billion

transistors [3]. While the reprogrammability of FPGAs allows fast realization and in-system debugging of a hardware design, thereby greatly improving designer productivity, this advantage is undermined if the CAD tool runtime is too long or the Quality of Results (QoR) is poor. Therefore, developing new CAD algorithms that can produce high-quality results for large designs in reasonable runtime is a key target for the FPGA industry.

Placement is a crucial stage in the FPGA CAD flow. Typically, it is the most time-consuming stage of the FPGA compilation flow [4], with a runtime of several hours or more for large designs. Moreover, it greatly affects the QoR of the entire flow. If the placement algorithm produces a solution that requires too much wiring, the router will fail—or take a long time—to find a solution, as routing resources in an FPGA are prefabricated and hence of limited capacity. Placement also greatly affects the final design speed as the best-case routing delays are determined by placement. Final optimization techniques, such as buffer insertion and gate resizing, that are common in ASIC CAD are not possible in FPGAs; therefore, a placement solution that forces slow wiring on the critical path cannot meet timing requirements regardless the time and effort expended later in the flow. FPGA placement is also a more constrained problem than ASIC placement because the locations where blocks of different types can be placed and the wiring patterns allowing interconnection of these blocks are both prefabricated.

Traditionally, three main techniques have been used to attack the placement problem: 1) partition-based; 2) analytical placement (AP); and 3) iterative improvement [especially simulated annealing (SA)] techniques. Partition-based techniques [5] use divide-and-conquer techniques to recursively partition a circuit and induce ever-smaller placement problems. These techniques can produce a solution in a short runtime, but they suffer from low QoR.

AP, on the other hand, approximates the placement goal as a smooth function and finds block locations that minimize this function under a set of constraints that are gradually refined to guide the solution to a mostly legal placement. Analytical techniques are used in several FPGA academic placers, such as Gplace [6] and elfPlace [7]. The most common goal in analytical placement for FPGAs has been to optimize wirelength and routability; however, the recent Liquid [8] and FTPlace [9] tools incorporate timing goals as well.

To fully optimize an FPGA design, analytical placers generally need a detailed improvement (i.e., refinement) step. These detailed placers are often implemented either as a low

Manuscript received 21 April 2021; revised 10 July 2021; accepted 24 August 2021. Date of publication 3 September 2021; date of current version 19 July 2022. This work was supported in part by the NSERC/Intel Industrial Research Chair in Programmable Silicon; in part by the Intel/VMware Crossroads 3-D FPGA Academic Research Centre; in part by Huawei; and in part by Google. This article was recommended by Associate Editor W. Hung. (Corresponding author: Mohamed A. Elgammal.)

Mohamed A. Elgammal is with the Department of Electrical and Computer Engineering, University of Toronto, Toronto, ON M5S 3G4, Canada, and also with the International Centre for Spatial Computational Learning (SpatialML), Toronto, ON, Canada (e-mail: mohamed.elgammal@mail.utoronto.ca).

Kevin E. Murray is with Cerebras Systems, Toronto, ON M5H 1J9, Canada (e-mail: kmurray@ece.utoronto.ca).

Vaughn Betz is with the Department of Electrical and Computer Engineering, University of Toronto, Toronto, ON M5S 3G4, Canada, also with the International Centre for Spatial Computational Learning (SpatialML), Toronto, ON, Canada, and also with the Vector Institute for Artificial Intelligence, Toronto, ON M5F 1M1, Canada (e-mail: vaughn@ece.utoronto.ca).

Digital Object Identifier 10.1109/TCAD.2021.3109863

temperature SA algorithm, or as greedy block moves (as in HeAP [10] and RippleFPGA [11]) that can be considered a zero temperature (quench) SA algorithm. The liquid placer combines analytic placement with SA to legalize the locations of “hard” blocks (e.g., RAMs and DSPs). The commercial Intel Quartus placer [12] is also a hybrid approach where AP is used to determine the initial placement solution and annealing is used to fine tune the placement.

SA is also used as a standalone placement algorithm for FPGAs, as it naturally handles the constraints on legal placement and the complex (and often discontinuous) routing delay versus net terminal location functions in FPGA architectures. For example, VPR [13] is a widely used academic place and route tool, which uses SA-based placement to adapt to a variety of FPGA architectures, and hence, is used to explore and evaluate FPGA architecture enhancements. VPR is also used as the production CAD flow to program both novel research FPGAs and some commercial FPGAs with an open-source flow [14]. SA placers are known to achieve high-quality results at the expense of long runtime.

As SA is the backbone of many placement approaches and the detailed improvement step of other ones, our work increases its efficiency by proposing new directed moves that can explore the solution space in reasonable runtime while preserving optimization stability. However, scheduling the use of different directed moves for different FPGA architectures, different circuits, and at different stages of the placement evolution is a challenging task. Consequently, we want our algorithm to “learn” the most effective perturbations using reinforcement learning (RL).

Using this technique, SA can be a framework unifying diverse placement improvement algorithms, such as simple move/swap, directed (smart) perturbation, minimizing a placement function for a part of the circuit (analytic techniques), or detailed placement of the critical path(s). Many of the ideas proposed in the different analytical placers [6], [8], [9] can be used as move types in this framework, with an RL agent choosing the appropriate move type at each optimization point.

To achieve this goal, we propose *RLPlace* that uses multiple types of directed moves to optimize both wirelength and critical path delay (CPD). *RLPlace* utilizes RL techniques to dynamically select the most effective move types throughout different placement stages, on different circuits, and targeting different FPGA architectures. Using these techniques, *RLPlace* helps the annealer rapidly and efficiently explore the large placement solution space, allowing SA to obtain high-quality results in reasonable runtime.

This article is an extended and refined version of our preliminary work presented in ICFPT 2020 [15]. We significantly enhance that work by proposing several new directed moves, placement checkpointing to reduce oscillation, adding more detailed analysis of the agent parameter selection process, and evaluating the algorithm effectiveness on a wider variety of FPGA architectures and benchmark suites.

Our contributions include as follows.

- 1) A new set of directed moves that make optimization more stable by avoiding oscillation.
- 2) A detailed analysis of the RL agent design choices.

- 3) A checkpointing technique that further reduces oscillation by periodically saving the best placement.
- 4) Evaluation of *RLPlace* performance on different FPGA architectures and design sizes.

## II. RELATED WORK

Prior research focused mainly on optimizing the runtime of SA in FPGA placement through two productive approaches: 1) parallel move generation and 2) directed search of the solution space.

One way to reduce SA runtime is to leverage multiple CPU cores. Ludwin *et al.* [16] performed multiple moves (placement perturbations) in parallel in the commercial Quartus placement algorithm, while preserving both algorithm determinism and achieving QoR equivalent to a serial annealer. An *et al.* [17] explored related techniques in the academic VPR place-and-route tool, and showed that speedups of  $5.9\times$  were achievable while maintaining quality and determinism. An *et al.* [17] showed that further parallelism could be exposed if the proposed moves were constrained to optimize different regions of the chip, obtaining higher speedups, but at a cost of 10% quality loss. Fobel *et al.* [18] achieved  $19\times$  speedup using parallel techniques on GPUs for nontiming driven placement. The techniques we explore in this article are complementary to these parallel techniques, as the more effective moves we create and select could also be performed in parallel.

A second technique to speed up SA is to use directed moves. Vorwerk *et al.* [19] proposed multiple strategies to create moves that more efficiently search the FPGA placement solution space for better timing and wirelength results. They calculated the effectiveness of each move and used this effectiveness to determine the probability of doing it in the future. Altera’s Quartus placer also uses more than ten different types of directed moves, but to our knowledge, no details about the exact directed move types are employed or how they are selected has been published [20]. ARBSA [21] is another SA-based placement engine that proposes “range-based” moves that limit the swap operations to a range of locations around the center of a net that connects to the moving block. Our work is in the same broad space as these prior studies, but we explore many different move types and develop new moves that reduce quality oscillations that we found to be common with previously proposed directed moves.

Recently, a variety of machine learning (ML) techniques have been applied to CAD problems. Alhyari *et al.* [22] used a convolutional neural network (CNN) to estimate the routability of a circuit placement. Al-Hyari *et al.* [23] proposed an adaptive FPGA placement algorithm that forecasts the final placement congestion based on features available early in placement using a convolutional encoder decoder. They used this estimated congestion to guide optimization decisions in order to reduce runtime.

To guide the annealer in choosing which type of move to make, we explore RL approaches that have not been previously considered in this domain. RL techniques have received significant attention recently due to their superior performance in many applications, such as website recommendation [24] and

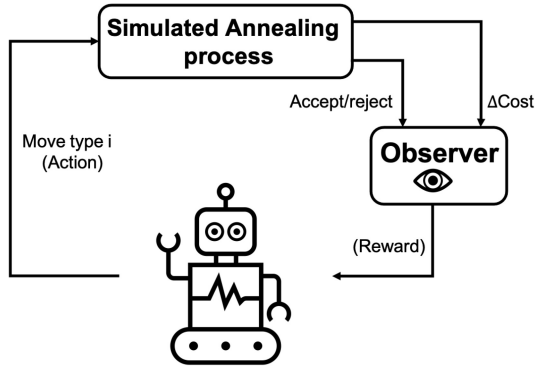


Fig. 1. RL problem formulation.

decision-making games (e.g., Go and chess [25]). RL has also recently been applied to several areas within electronic design automation (EDA): floorplanning [26], [27], routing [28], and transistor sizing [29]. A survey about the use of RL in EDA can be found in [30].

In FPGA placement, Murray and Betz [31] proposed using a simple single-state RL agent to choose which type of block to move during the anneal, but still always moved the block randomly. They showed that their pareto-optimal results outperformed VPR 8 in terms of QoR achieved in a given runtime. In this work, we explore a much wider variety of moves, allowing much more freedom and hence, more effective optimization by the RL agent.

### III. BACKGROUND

#### A. Reinforcement Learning

RL is an ML approach that has led to high-quality results in a wide variety of domains [32]. RL techniques employ a software agent that, given the current environment state  $s_t$ , chooses one of multiple actions to maximize the total expected reward. The agent works in a closed loop as shown in Fig. 1. The agent interacts with an environment at step  $t$  by performing an action  $a_t$  from a set of the available actions  $A$ . Then, the agent interprets how the environment reacts to the chosen action by mapping the environment's output  $O_{t+1}$  to the reward of this action  $r_{t+1}$  and determining the state of the next timestamp  $s_{t+1}$  from a set of the available states  $S$ . Any action chosen will not only affect the immediate reward but also all the upcoming rewards and states. RL techniques frequently rely on an *action value* function  $Q(s, a)$  to predict the immediate and future expected reward if action  $a$  is chosen while the environment is in state  $s$ .

The two main considerations for any RL algorithm are as follows.

- 1) *Action Value Estimation*: How  $Q$  is estimated from the agent's experience.
- 2) *Action Selection*: How  $Q$  is used to select the next action.

A well-known special case of the RL problem occurs when there is only a single state, and is called the  $K$ -armed bandit problem [32]. In this case,  $Q$  becomes a function of the actions only:  $Q(a)$ .

---

#### Algorithm 1: SA Algorithm

---

**Input:** Initial Placement  $P_{init}$ , Initial temp  $T_{init}$ ,  $M$  moves per temp.  
**Result:** An optimized placement  
 $P \leftarrow P_{init}$ ,  $T \leftarrow T_{init}$   
**while**  $\neg(\text{ExitCondition}(P, T))$  **do**  
  **for**  $1 \dots M$  **do**  
     $P' \leftarrow \text{GenerateMove}(P)$   
     $\Delta\text{cost} \leftarrow \text{EvaluateMove}(P, P')$   
    **if**  $\text{AcceptedMove}(\Delta\text{cost}, T)$  **then**  
       $P \leftarrow P'$   
    **end**  
  **end**  
   $T \leftarrow \text{UpdateTemp}(T)$   
**end**  
**return**  $P$

---

One method to estimate  $Q$  values is to store the estimated reward of taking a particular action  $a_t$  in a table. After performing action  $a_t$  and receiving reward  $r_{t+1}$  the  $Q$  value is updated as

$$Q(a_{t+1}) = Q(a_t) + \alpha \times (r_{t+1} - Q(a_t)) \quad (1)$$

where the term  $r_{t+1} - Q(a_t)$  represents the error between the agent's estimate and the actual reward, and  $\alpha$  is a parameter that chooses how large a step the new action value estimate takes to reduce this error.

When the number of actions and/or states becomes large, storing  $Q$  in a table becomes infeasible and it must be approximated. Deep RL (DRL) uses a deep neural network as the  $Q$  function approximator, and the agent's experience is used to tune the neural network weights.

#### B. Simulated Annealing Placement

SA [33] is a metaheuristic that mimics the annealing process used to form strong metal alloys; it is one of the most commonly used approaches to solve the placement problem. SA starts with an initial solution and repeatedly generates perturbations (or *moves*) based on the current solution. It estimates the change in the solution cost to decide if the move is accepted (updates the placement) or not. At the beginning of optimization, the *temperature* is high (corresponding to a high energy state in metal annealing) and SA has a high probability of accepting any move. As optimization progresses the temperature drops, accordingly, the probability of accepting a move that increases the placement cost also drops. When the temperature becomes zero, the algorithm becomes greedy: only moves that decrease cost are accepted. This technique can escape local minima as it accepts some moves that increase the cost (i.e., hill climbing) when the temperature is above 0. However, it needs a large number of moves to explore large solution spaces. Algorithm 1 shows the SA procedure. The QoR and runtime of SA are strongly affected by both the annealing schedule, which determines how the temperature is decreased in `UpdateTemp`, and the usefulness of

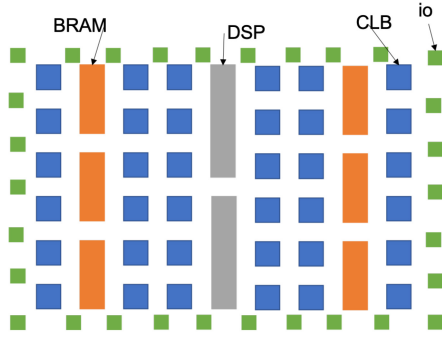


Fig. 2. FPGA location grid.

the placement perturbations proposed by the `GenerateMove` function.

In FPGA placement, a perturbation (move) means changing the location of some netlist blocks, usually by moving them to empty locations or swapping them with the blocks already at those locations. As shown in Fig. 2, FPGAs are heterogeneous, with different locations in the FPGA resource grid being able to accommodate different block types, such as IOs, CLBs, DSPs, and BRAMs; legal moves can only move a block of a certain type to a grid location of the proper type. As well, blocks in some structures such as carry chains must be kept in fixed relative orientations to be routable, so such blocks are moved as a unit to maintain legality. In VPR, primitives (such as LUTs, FFs, multipliers, and RAM slices) are first clustered together into logic, DSP, RAM, and other *cluster-level* blocks. The most detailed legality constraints in FPGAs concern what makes a legal cluster-level block, and VPR’s clustering engine supports very general interconnect, control signal, mode, and other legality restrictions, thereby increasing the architecture generality of the tool. RLPlace builds on this flow, and similarly, performs placement of the cluster-level blocks.

VPR [13] uses simple moves that move or swap blocks randomly within a certain distance. However, there are more powerful moves that exploit information about the netlist and the structure of the problem to move blocks toward their optimal locations, as we detail in the next section.

#### IV. DIRECTED MOVES

As stated earlier, the main limitation of SA-based placement is its long runtime to converge to a high-quality solution. This long runtime has been exacerbated by the continual growth in FPGA device capacity, and hence, the solution space size. A productive technique to increase the speed of SA convergence is to propose intelligent directed, perturbations, rather than purely random ones.

Ideally, directed moves should not only propose perturbations that are likely to be useful but also require low CPU time so that they can be run hundreds of thousands or millions of times to search the design space thoroughly. Hence, there is a tradeoff between having a very intelligent move that has a high probability of reducing the CPD and/or wirelength and the time spent to calculate and propose this type of move. Although directed moves can enhance timing and/or

wirelength, if care is not taken in their design, the large perturbations they propose can cause harmful oscillation in timing performance by modifying critical paths too aggressively.

In the following sections, we discuss the move engines we proposed in RLPlace and how they balance these competing goals.

##### A. Random Move

In our work, we include the original VPR 8 move, which we call the “Random Move (Rand.)” This move chooses a block (or group of blocks that must move as a group, such as a carry chain) randomly from the netlist. Then, it chooses a random location of the same type (CLB, DSP block, etc.) within a *range limit* around the original location in the device. If the location is empty, it moves the block to this location. If another block was already placed in the proposed location, the two blocks are swapped. Finally, it estimates the change in timing (based on best case routing delays and the timing criticality of the affected connections) using Tatum—a full featured static timing analysis engine that supports multiclock timing analysis and timing exceptions [34]—and wirelength (based on the half-perimeter of the net bounding boxes, or HPWL) costs to assess the move and determine whether to accept or reject it.

The range limit in this move is initially large, allowing blocks to be moved large distances (usually any legal location in the chip) early in the anneal. As the anneal progresses, this range limit parameter shrinks, at a rate determined by the fraction of moves being accepted, in order to gradually propose more local perturbations that have a reasonable likelihood of being accepted as the temperature drops. Late in the anneal, the range limit is one unit, meaning that blocks can only move to the closest compatible locations in the  $x$  and  $y$  directions. This adaptive range limit technique has been shown to improve results quality for a given CPU time [35].

##### B. Bounded Median Move (B.Med.)

This move was initially proposed by Vorwerk *et al.* [19] and enhanced in our earlier work [15]. However, the large perturbations it proposes lead to oscillations in wirelength and CPD, especially for large designs, that degrade final result quality. Hence, we have created a new move with a similar goal that achieves superior results by damping these oscillations.

This move mainly targets wirelength minimization by moving a block ( $b_i$ ) into its *median region*, which is the  $(x, y)$  range of locations that will minimize the HPWL of the nets connected to it.

First, the median region is calculated using Algorithm 2. The main differences between this algorithm and the one proposed in [19] are that in the *get\_bounding\_box* function, we are using an incremental bounding box calculation technique similar to [35]. Moreover, we exclude nets with fanout higher than *high\_fanout\_limit* terminals to speed up the calculation. Empirically, we found that excluding nets with fanout higher than 10 has no effect on the QoR but greatly improves runtime.

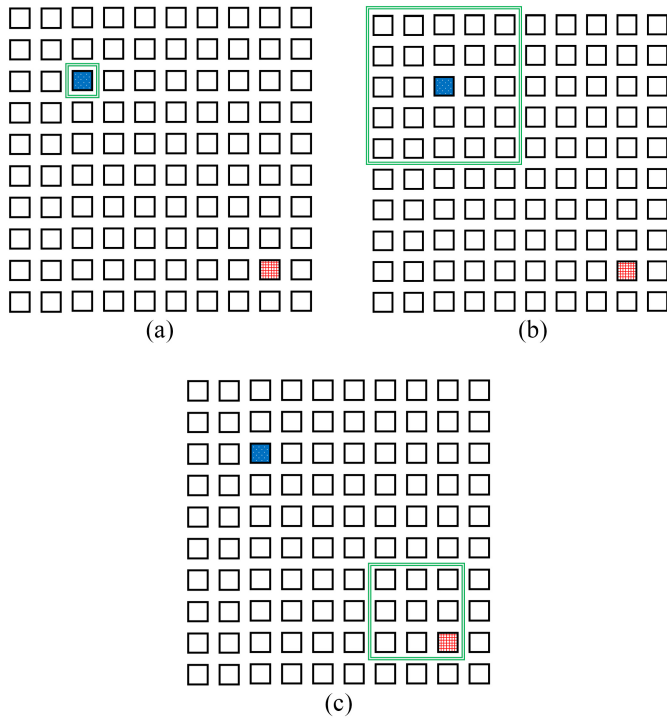


Fig. 3. Comparison of the different location selection techniques. In all cases, the range limit is 2. The red crossed box is the original block location, the blue-dotted box is the center of the calculated median region, and the green box highlights the region to which we will move the block. (a) No range limit. (b) Target-centered range limit. (c) Bounded range limit.

---

**Algorithm 2: Median Move**


---

```

Choose a random block  $b_i$ 
Define empty vectors  $X_{coords}, Y_{coords}$ 
for  $P_i$  in  $block\_pins(b_i)$  do
     $n_i$  is the net connected to pin  $P_i$ 
    if  $n_i$  is not a high fanout net then
         $x_l, x_r, y_l, y_h = get\_bounding\_box(n_i)$ 
        push  $x_l, x_r$  in  $X_{coords}$ 
        push  $y_l, y_h$  in  $Y_{coords}$ 
    end
end
Sort  $X_{coords}, Y_{coords}$ 
 $x_1, x_2 =$  the two middle elements in  $X_{coords}$ 
 $y_1, y_2 =$  the two middle elements in  $Y_{coords}$ 
Define the median region to be  $x_1 < x < x_2, y_1 < y < y_2$ 
Choose a move location for the block given the
calculated median region.

```

---

Vorwerk *et al.* [19] moved to a randomly chosen location within the median region, but we found that for heterogeneous FPGAs, it was better to move to the legal location that is closest to the center of the median region. This is shown in Fig. 3(a) where the moving block is highlighted in red, the center of the median region is the blue location, and the green box is the range of locations to which we can move the block. This formulation is restrictive as there is only one proposed move location for the red block (the center of the median region); this could lead to proposing

the same move repeatedly and not exploring the solution space well.

Our second choice was to use range limiters to allow more move diversity. This approach allows a block to be moved to a random location in a range around the center of the median region as shown in Fig. 3(b), where the range limit in this case is 2. We call this technique the target-centered range limit. This range limit value evolves during the anneal, dropping from *max\_range\_limit* units (set to 3 in our later results) in any direction around the center of the median region at the start of the anneal to 1 unit late in the anneal. This range is applied to the compressed device grid of the block type [13], which enables sparse blocks (e.g., RAMs) to move to a different column (as the adjacent column is 1 unit away in this space) even late in the anneal.

However, we found that using this location selection technique causes oscillations in the placement quality late in the anneal, particularly for large designs. As shown in Fig. 3(b), the target-centered range limit technique allows the moving block (the red box) to be moved to a distant part of the fabric (any location in the green box) even with a small range limit (equals to only 2 in this example) when the center of the median region (the blue box) is far from the current block location. Allowing such large moves late in the anneal causes placement quality—especially the timing performance—to oscillate.

To address this issue, we propose the *bounded range limit*, a new range limit algorithm that tries to achieve stable optimization late in the anneal while still exploiting the intelligent wirelength targeting of median moves. Early in the anneal, we use the target-centered range limit algorithm where a block is moved to a range around the center of the median region. Then, later in the anneal, we move the range to be around the current location of the block but directed toward the center of the median region as shown in Fig. 3(c).

This technique achieves two benefits.

- 1) It allows blocks to be moved anywhere in the device early in the anneal to rapidly optimize wiring.
- 2) It limits the moves later in the anneal to be near the current block location while using the median region calculation to direct the motion of the block toward wirelength minimizing locations.

To evaluate these different variants of the median move, we evaluate the QoR achieved when the placement algorithm uses only the bounded median and random moves, with an equal probability of each. Fig. 4 shows the CPD of one of Titan23 benchmarks (bitcoin\_miner), which is one of the largest benchmarks in this suite with more than one million primitives. The graph shows the estimated CPD after each temperature during the anneal. It is clear that using no range limit and only moving the block to the center of the median region (the green line) takes many more temperatures (more runtime) to converge to a CPD of 9.5 ns. Using the target-centered range limit algorithm (the red line) helps the annealer to converge much faster but causes many oscillations in the CPD late in the anneal. It converges to a solution that has CPD equal to 9.4. However, using the bounded range limit technique (the blue line) greatly reduces the QoR oscillations, especially late in the anneal, and

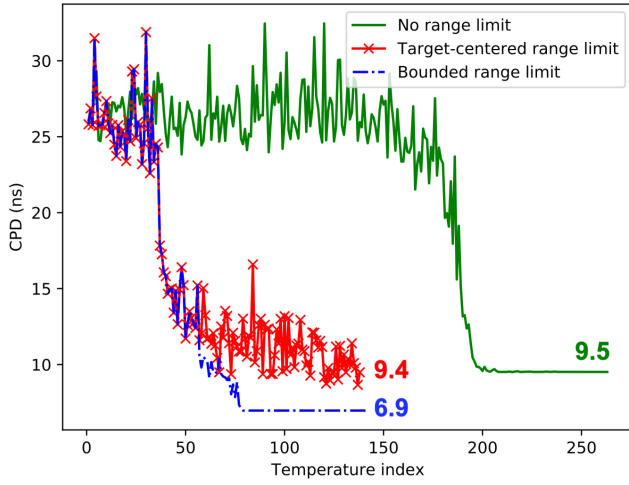


Fig. 4. Comparison of the effect of different median move variants on estimated CPD throughout the anneal of the bitcoin\_miner Titan23 benchmark.

achieves a CPD of 6.9 allowing an operating frequency 36% higher than that achieved with the target-centered range limit. On the wirelength side, the no range limit technique achieves a total (HPWL) estimated wirelength of 28.7e6 while both the target-centered and the bounded range limit techniques achieve wirelengths of 9.51e6 and 9.46e6 (only 0.33 $\times$  of the WL achieved by no range limit). Similar trends are observed on most of the designs and hence, we use the bounded range limit technique for median moves throughout the remainder of this work.

### C. Bounded Edge-Weighted Median Move (B.E.W.Med.)

Vorwerk *et al.* [19] proposed constructing a median region as described in the prior section, but then weighting (shifting) the coordinates by net criticality in an attempt to improve circuit timing. The criticality of any connection is a value  $\in [0, 1]$  where connections with criticality 1.0 have no slack, and those with criticality 0.0 have a large positive slack [36]. We found this formulation did not achieve good results. Instead, we developed a new formulation that constructs target regions from criticality weighted *edges* of the net bounding boxes; we call this move edge-weighted median and describe it as follows.

Fig. 5 shows a bounded edge-weighted median move. We are moving one block (the blue square) connected to two nets (the green and red dotted lines). The sinks of each connection have their timing criticality values labeled. The bounding box of each net is outlined by the solid colored box with the same color as its net. In the median move,  $x_1, x_2, x_3$ , and  $x_4$  are all weighted equally and the same for  $y_1, y_2, y_3$ , and  $y_4$ . In the edge-weighted median move, however, each coordinate is weighted by a different factor; that is, it is inserted in the list of coordinates from which we select the median a different number of times. In this example,  $x_1$  will be weighted by one, following 2 and given that the criticality of the connection from the red net driver to the moving block is (0.1):

$$\text{weight} = \text{ceil}(10 * \text{criticality}). \quad (2)$$

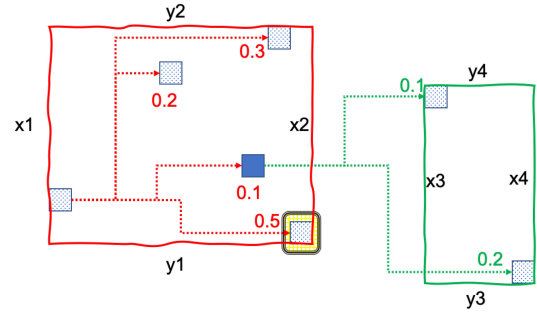


Fig. 5. Bounded edge-weighted median move example. The solid box is the moving block, the dotted boxes are the fanin and fanout blocks, the numbers are the timing criticalities of various connections, dotted lines are the connections, and the solid line represents the bounding box of each net. The final bounded edge-weighted location is the yellow colored region inside the black box.

$x_2$  will be weighted by 5 (inserted five times in the list) as this edge of the red net bounding box is determined by a pin on a connection with timing criticality of 0.5. Following the same strategy,  $x_3$  and  $x_4$  will be weighted by one and two correspondingly, and  $Y_{\text{coords}}$  are computed similarly. We then calculate the median of the weighted values to extract the bounded edge-weighted median region, which, in this case, is a single location:  $(x_2, y_1)$ . Bounded edge-weighted median can simultaneously optimize timing and wirelength by shrinking net bounding boxes with a preference to move closer to timing-critical pins that are setting the net bounding box limits. This helps timing in two ways: directly, by moving the driver of a critical connection toward the critical sink, or indirectly, by moving terminals of a net toward the most critical terminal making it easier to improve timing to this group of terminals in a later move.

As with the bounded median move, we used the bounded range limit algorithm to turn this target region into a proposed move location in order to achieve stable optimization with fewer quality oscillations.

### D. Bounded Centroid Move (B.Cen.)

The bounded centroid move is inspired by analytic placers, which model net connections as springs and calculate the point of force equilibrium. It is also similar to the force-directed (FD) moves used in Hentschke's greedy perturbation placer [37] for ASICs. For a chosen block, this move calculates the forces/weights acting on the block based on its connections to other blocks. A net driver is attracted to all its sinks (if the moving block is the net driver, there will be multiple edges to all the net sinks), while a net sink is only attracted to its driver (if the moving block is a net sink, there will be a single edge to the net driver).

We then calculate the zero-force  $x$ -position  $X_c$  using (3). The summations are over the input and output nets of the block,  $x_{d(i)}$  is the  $x$  location of the driver of net  $i$ ,  $\text{sinks}(i)$  is a list of the sink pins of net  $i$ , and  $W_j$  is the weight of the connection between the moving block and the pin  $j$ .  $Y_c$  is calculated similarly. We weight each connection to the moving

block by 1

$$X_c = \frac{\sum_{i \in \text{in nets}} x_{d(i)} * W_{d(i)} + \sum_{i \in \text{out nets}} \left( \sum_{j \in \text{sinks}(i)} x_j * W_j \right)}{\sum_{i \in \text{in nets}} W_{d(i)} + \sum_{i \in \text{out nets}} \left( \sum_{j \in \text{sinks}(i)} W_j \right)}. \quad (3)$$

We found that moving only to the single location calculated in this way is too restrictive: there may be no legal location for the block type at that spot, or we may propose too many moves to the same spot and not explore the space adequately. Consequently, we use the bounded range limit algorithm around the calculated  $(X_c, Y_c)$  location to choose a target location.

#### E. Bounded Weighted Centroid Move (B.W.Cen.)

The bounded Weighted centroid move is similar to the bounded centroid move, but weights each connection by its criticality,  $W_i = \text{Crit}_i$  in order to preferentially shorten timing-critical connections.

#### F. Critical Random Move (C.R.)

Another move that can improve the CPD is the critical random move. This move is similar to the random move described in Section IV-A but it only chooses from a subset of blocks to move: those with at least one connection with timing criticality  $> \text{high\_crit\_threshold}$ . It moves this block to a compatible random location in a range around its original location. This move is combined with frequent timing analysis in [34] to improve timing.

An advantage of this move is that it requires minimal extra computation, which means it is fast to propose and evaluate. At the same time, it focuses more on moving highly critical blocks, increasing the probability of finding a good placement for the speed-limiting parts of a circuit. For the results of this article, we set  $\text{high\_crit\_threshold}$  to 0.7.

#### G. Quasibounded Feasible Region Move (QB.FR.)

Chen and Cong [38] defined a feasible region (FR) where a block can be placed to minimize the CPD, and Vorwerk *et al.* [19] used the same idea in their MPD move. Similarly, we propose an FR move, which moves a highly critical block to a random location inside its FR. First, we choose a random block with highly critical connections to move, as in the critical random move. Then, we identify the highly critical inputs to the block and the most critical output. Finally, we calculate the FR using the algorithm proposed in [38].

We first try to move the block to a random location in the calculated FR. If we are unable to find a legal location in the region, we extend the region using the bounded range limit algorithm discussed earlier. Accordingly, we call this move a quasibounded FR move. While this move takes more computation than the critical random move, it directly minimizes the delay of critical connections.

### V. REINFORCEMENT-LEARNING-BASED PLACEMENT

After defining and implementing the directed moves in Section IV, we next determine how to use them effectively. In particular, we need to answer: when should each move

TABLE I  
RL AGENT ACTIONS AND STATE SPACE

| Space           | Values                                                       |
|-----------------|--------------------------------------------------------------|
| Actions ( $A$ ) | {Rand., B.Med., B.Cen., B.W.Cen., B.E.W.Med., C.R., QB.F.R.} |
| State ( $S$ )   | { $S_0, S_1$ }                                               |

type be used? what should be the probability of each move occurring? should these probabilities be fixed for all designs and FPGA architectures? should they even be fixed throughout the annealing process?

We believe the answer will be highly situational. For instance, the most effective moves will depend on the netlist we want to place, the current placement progress, and the target architecture. Due to the large number of factors that affect the most effective moves and with a multiplicity of available moves, it would be challenging or impossible for a human expert to create a fully tuned schedule of moves to propose at different points in time.

As a result, we leverage an RL agent to *automatically* adapt the probability of using each move according to each situation. In the next sections, we discuss our formulation of move type selection into an RL problem and how we designed the RL agents.

#### A. Problem Definition

As we discussed in Section III, there are two techniques used in RL to estimate the state action value  $Q(s, a)$ : 1) tabular methods and 2) function approximation methods (DRL). The choice between these approaches depends mainly on the problem state and action spaces.

Murray and Betz [31] used a tabular method with a single state and an action space of moving different types of blocks randomly. We expand the action space to the larger variety of move types detailed in Section IV and also introduce multiple states that represent the stage of the annealer. As shown in Table I, there are seven actions corresponding to the seven moves discussed in Section IV. The state space comprises two states, which represent the stage of the annealing process.  $S_0$  represents the earlier part of the anneal, when large changes are being made and the focus is on obtaining a good global placement that optimizes wirelength and has reasonable overall timing. Late in the anneal, there is more visibility into the true critical paths of the design, and additional focus on timing optimization is appropriate. Consequently, in the first state, only the first four actions are available as they primarily target wiring and coarse timing optimization. We determine that we have moved into the second ( $S_1$ ) state by monitoring the rate of temperature update in VPR's adaptive annealing schedule; when the new temperature is at least 30% smaller than the prior temperature, placement is no longer performing much hill climbing, indicating we are in  $S_1$ . In the  $S_1$  state, the RL agent will select amongst all seven move types, including the final 3 in Table I (bounded edge-weighted median, quasibounded FR, and critical random), which focus more on timing optimization.

TABLE II  
EXPERIMENT SETS DESCRIPTION

| Benchmarks | Arch.           | num of primitives  |
|------------|-----------------|--------------------|
| Large VTR  | k6_frac (N=10)  | 19,549 - 166,678   |
| Titan23    | Stratix IV-like | 90,778 - 1,061,829 |

Having only two states makes it feasible to use tabular methods to estimate state–action values, and is similar to having two  $K$ -armed bandit agents (one per state). In detail:

- 1) the agent chooses an action  $a_t \in A$  (a move type);
- 2) the move is evaluated, blocks are swapped, and the change of the placement cost is calculated as  $\Delta bb_{\text{cost}}$  (the normalized change of wirelength cost),  $\Delta t_{\text{cost}}$  (the normalized change of timing cost), and  $\Delta \text{cost}$  (the total change of cost) [39];
- 3) the annealer decides whether to accept or reject the move;
- 4) the agent receives a reward  $r_{t+1}$  to update the  $Q$  values based on the calculated  $\Delta \text{costs}$ , thereby influencing future action selection.

In our case, the problem is affected by both static characteristics, such as circuit structure and FPGA architecture, and dynamic characteristics, such as the types of moves that are most effective and the current placement quality, which are evolving throughout the optimization process. Hence, our problem is nonstationary and the agent should pay more attention to more recent rewards as they better reflect recent changes in the environment. Consequently, as in [31], we set  $\alpha$  (in 1) according to the exponentially weighted formula as follows:

$$\alpha = 1 - e^{\log(\gamma)/M} \quad (4)$$

where  $M$  is the number of moves per temperature, and  $\gamma$  is a hyperparameter that controls the fraction of weight given to moves which occurred  $> M$  moves ago (effectively controlling the length of the agent’s memory).

### B. Methodology

To design the RL agent and choose its parameters, we took an empirical approach. We implemented RLPlace in the VTR framework [13] and used different benchmark suites and FPGA architectures as shown in Table II for tuning and testing. For the VTR benchmark suite [13], we excluded small circuits with fewer than 10K primitives and targeted the flagship VTR architecture k6\_frac\_N10\_frac\_chain\_mem\_32K\_40nm.xml [13], which is a realistic FPGA architecture with ten 6-input fracturable LUTs and 20 FFs per logic block, along with RAM, DSP, and IO blocks. To further study how performance scales with increasing design size and the robustness of our results on different FPGA architectures, we also used the Titan23 benchmark suite [4], which targets a model of the commercial Stratix IV FPGA architecture [13], [40].<sup>1</sup>

<sup>1</sup>We exclude the very large Gaussian\_blur design from most of our tuning results due to its high runtime, but it is included in the final evaluation of the RLPlace algorithm.

CPU runtime and QoR are both important metrics when evaluating and comparing placement algorithms. In most placement algorithms, there are some parameters that can be tuned to control the runtime/QoR tradeoff. This is useful as the low runtime results are important earlier in the design process to enable rapid iteration, while the high runtime (higher quality) results are important in the final design stages. Hence, the entire runtime/QoR tradeoff curve of a placement algorithm is of interest. To obtain a curve that represents this tradeoff, we varied the number of moves per temperature parameter ( $M$  in Algorithm 1). At each runtime point, we evaluate two QoR parameters: 1) the estimated wirelength to route the placement (based on VPR’s fanout-adjusted HPWL metric) and 2) the estimated CPD that would result after routing on the target architecture. Both these metrics correlate well to the postroute values but are faster to compute as they require only a placement. We validated that same results are achieved for the final (postroute) wirelength and CPD.

To avoid CAD noise, overall results are first averaged over five different seeds for the VTR benchmark results and three different seeds for the Titan23 benchmarks and then geometrically averaged over all circuits in the suite.

### C. Exploration Versus Exploitation

Once the agent updates the  $Q$  values, it selects the next action to perform. The agent faces either of two contradictory choices.

- 1) Exploit the optimal action from its knowledge (i.e., the greedy action). This selection will maximize the immediate reward.
- 2) Explore other nongreedy actions to improve its estimation of the  $Q$  values.

This issue is particularly challenging in nonstationary problems, where the agent’s previously learned best choice may become suboptimal later. There are many techniques to overcome this challenge [32], such as  $\epsilon$ -greedy and softmax formulations.

1)  $\epsilon$ -Greedy: In most cases, the agent will select the greedy action (the action with the highest  $Q$ ). However, for a small fraction  $\epsilon$ , it randomly selects one of the other (i.e., nongreedy) actions to explore. This technique is simple, easy to compute, and stable. However, one drawback is that it deals with all nongreedy actions equally. In particular, moves with high  $Q$  values (i.e., perceived reward) near those of the greedy action will be chosen with the same probability as moves with very low  $Q$  values.

2) Softmax: In softmax, the agent chooses moves proportionally to their state–action values ( $Q$ ). Hence, the greedy action would be more likely to be chosen, and other near-greedy actions will have comparable probabilities. Applying the common softmax formula to calculate the probability for each action yields

$$P_a = \frac{e^{Q(a)}}{\sum_{i=1}^k e^{Q(i)}}. \quad (5)$$

However, as the reward values are too small (in the range of  $1e-9$ ), we used the Boltzman softmax implementation instead

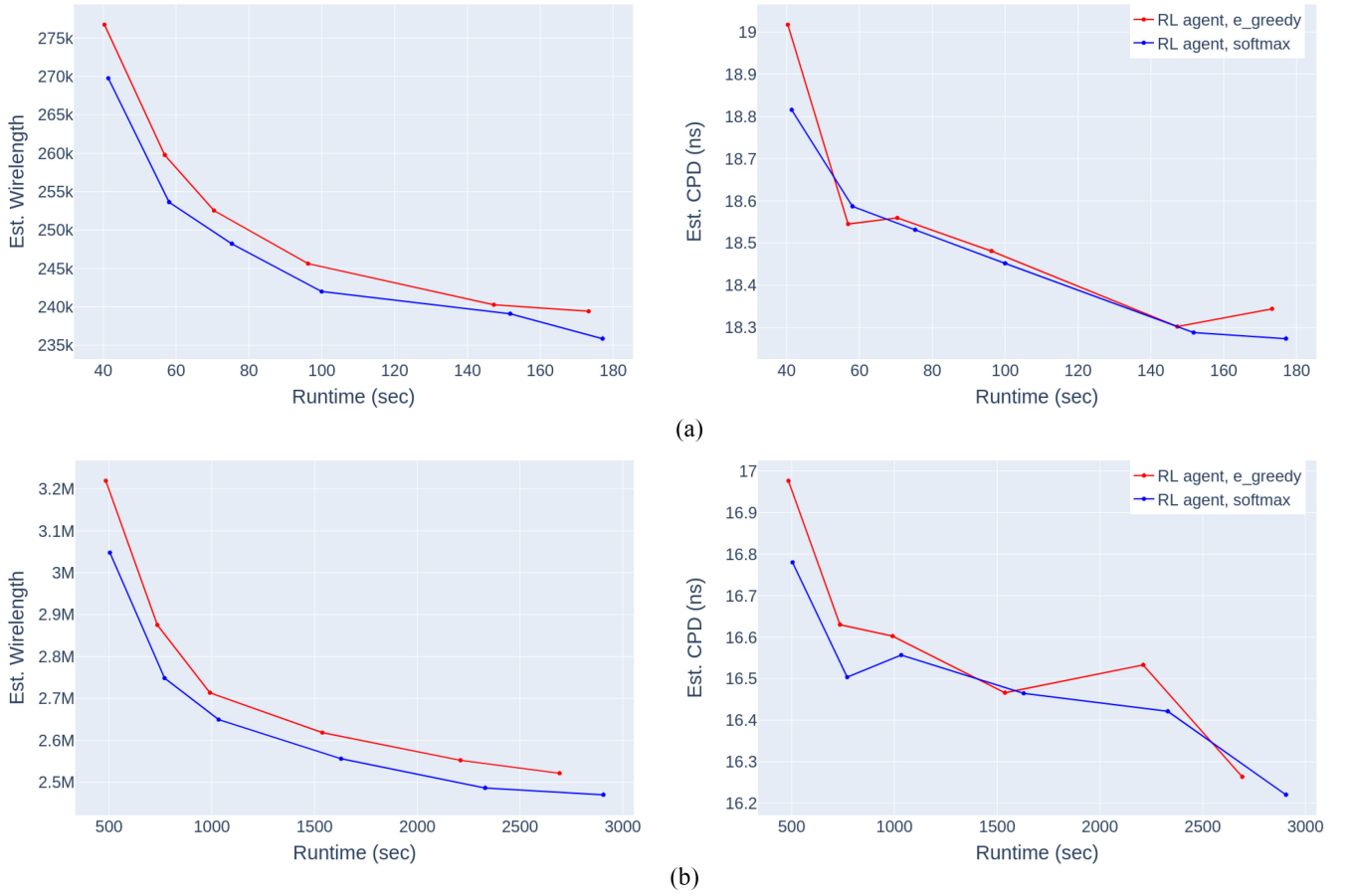


Fig. 6. Comparison of different RL exploration-exploitation techniques at different runtime/quality points. (a) VTR Benchmark suite, 5-seed average. (b) Titan23 Benchmark suite, 3-seed average.

to sharpen the exponential function as shown in (6), where  $\beta$  is a parameter that controls the sharpness of the exponential function [41]

$$P_a = \frac{e^{\beta \cdot Q(a)}}{\sum_{i=1}^k e^{\beta \cdot Q(i)}}. \quad (6)$$

To force exploration of all actions, we include a minimum probability for all actions ( $\alpha$ ), which enables exploration and is particularly important early in the anneal

$$P_a = \frac{\max(e^{\beta \cdot Q(a)}, \alpha)}{\sum_{i=1}^k \max(e^{\beta \cdot Q(i)}, \alpha)}. \quad (7)$$

To evaluate which technique would fit better to our problem, we implemented both:  $\epsilon$ -greedy and Softmax-like (7) and tested them on both the benchmark suites listed in Table II. Fig. 6 shows the comparison between those techniques at different runtime points. We pick the best performing parameters for each algorithm for a fair comparison. These results are using the *WLbiased\_runtime\_aware* reward function that will be detailed in Section V-D, but the trends are similar for other reward functions as well.

Although softmax-like needs more computation compared to  $\epsilon$ -greedy, it outperforms  $\epsilon$ -greedy at most runtime points, achieving less wirelength and/or less CPD at the same runtime. This occurs because multiple types of moves are productive

at a certain placement state and Softmax allows exploitation of multiple move types that are performing well. Hence, we prefer Softmax-like and it is the formulation used for the remainder of this article.

#### D. Reward Function

To help the agent interpret the environment's output, we need to define a *reward function*, which translates the output  $O_{t+1}$  into a reward  $r_{t+1}$ . The agent then tries to maximize its cumulative reward. Crafting a good reward function is important, since it is the only feedback the agent receives to adjust its behavior.

To capture our intent (a high-quality optimization result in reasonable runtime), we consider the change of the placement cost ( $\Delta\text{cost}$ ), which includes both timing and wirelength components. As the placement problem is a minimization problem but the agent's target is to maximize its reward, we use the negative of the cost change in any reward function we propose.

The first reward function we consider [31] is

$$r_t = -\Delta\text{cost}. \quad (8)$$

This reward function (*basic* reward) seems to match our intent of minimizing cost. However, it penalizes the agent with

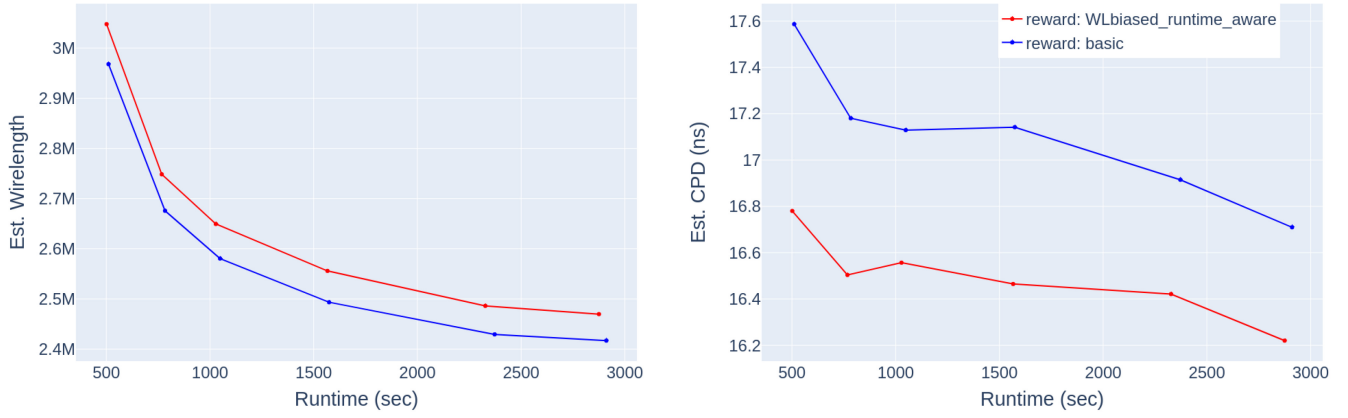


Fig. 7. Comparison of different RL reward functions at different runtime/quality points on the Titan23 benchmark suite, 3-seed average.

a large negative reward for moves that increase the cost (hill climbing moves), which makes it easier to get trapped in a local minimum [31]. Hence, we also consider the following reward function:

$$r_t = \begin{cases} -\Delta\text{cost}, & \text{if } \Delta\text{cost} < 0 \\ 0, & \text{otherwise.} \end{cases} \quad (9)$$

However, while the above reward functions capture the goal of generating high-quality placements, they do not capture any metric of the runtime to a solution. To make the agent runtime aware, we propose a reward function of

$$r_t = \begin{cases} \frac{-\Delta\text{cost}}{t(i)}, & \text{if } \Delta\text{cost} < 0 \text{ for move type } i \\ 0, & \text{otherwise} \end{cases} \quad (10)$$

where  $t(i)$  is a static normalizing factor, which represents the average time consumed to evaluate moves of type  $i$ . We pre-calculate  $t(i)$  by averaging the runtime of each move type over many circuits. With this reward function, the agent prefers the move type that maximizes the rate of quality improvement, thereby minimizing runtime.

Moreover, we proposed a reward function that includes the different components of the change of the placement cost ( $\Delta\text{bb}_{\text{cost}}$ ,  $\Delta\text{t}_{\text{cost}}$ ) [39] as shown in

$$r_t = \begin{cases} \frac{-1}{t(i)} \left( (0.5 + R_s) * \Delta\text{bb}_{\text{cost}} + (1 - R_s) * \Delta\text{t}_{\text{cost}} \right), & \text{if } \Delta\text{cost} < 0 \\ 0, & \text{otherwise} \end{cases} \quad (11)$$

where  $R_s$  is a *reward shaping* factor that controls the weights of each component of the change of the placement cost adding a flexibility to give them different weights. The optimal value for this parameter was empirically found to be 0.4, which gives more weight to WL. Hence, we called this reward function the *WLbiased\_runtime\_aware*.

Fig. 7 shows the comparison between the basic reward function (8) and the *WLbiased\_runtime\_aware* one (11) on the Titan23 benchmarks suite. It shows that both are reasonable choices. While the *basic* reward optimizes wirelength well, it is less robust for timing optimization as it avoids most of hill climbing moves. Other reward functions (9), (10) achieve a similar wirelength as (11). However, (11) attains more stable timing results over the different runtime points. Hence, we

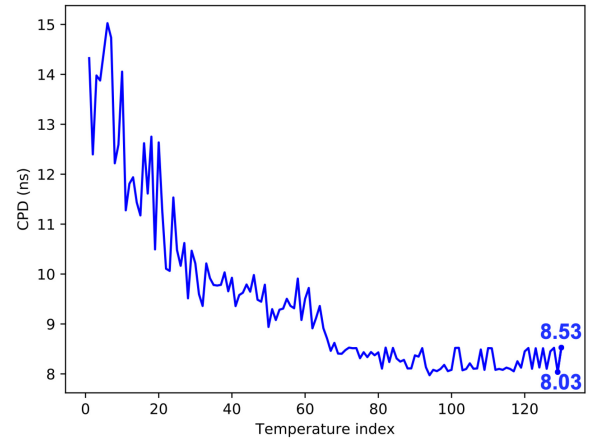


Fig. 8. Achieved CPD at each of the annealing temperatures for one of Titan23 Benchmarks (*neuron*).

prefer the *WLbiased\_runtime\_aware* and use it for the rest of this article.

#### E. Placement Checkpointing

As we discussed earlier in Section IV, we took performance oscillations into consideration while designing and implementing the proposed directed moves. However, oscillations are still not fully removed, especially on the timing/CPD side.

Fig. 8 shows the CPD for one of Titan23 benchmarks (*neuron*) over the different temperatures. The figure clearly shows that the oscillations on the CPD led to some timing performance degradation. For example, in this case, the placer was able to achieve a CPD of 8.03 ns at one point during optimization. However, at the end of the anneal, the placement has a higher CPD of 8.53 ns and the same wiring cost as the earlier and faster solution.

To overcome this issue, we *checkpoint* the best placement solutions. This checkpoint can be retrieved at the end of the anneal if the final achieved placement quality is worse than the checkpoint quality. To minimize the overhead of using this

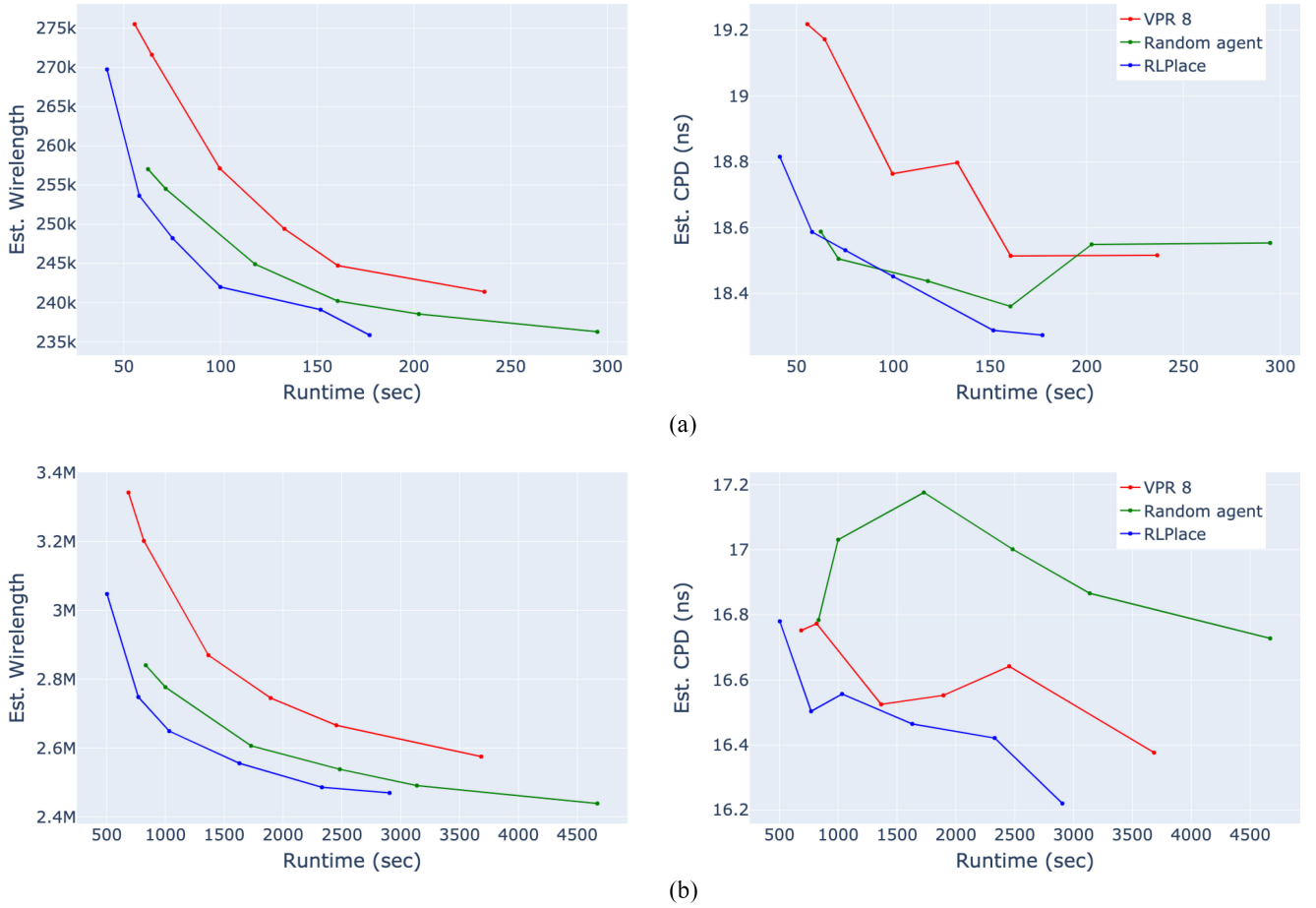


Fig. 9. Comparison of RLPlace, VPR 8, and a random agent with directed moves on different quality runtime tradeoff points. (a) VTR Benchmark suite, 5 seed average. (b) Titan23 Benchmark suite, 3 seed average.

technique, we start taking checkpoints late in the anneal when the placer has almost converged to a good solution.

Adding this checkpointing technique to *RLPlace* helped to achieve  $0.94\times$  CPD on average compared to *RLPlace* without checkpointing when tested on the Titan23 benchmarks with default options. This improvement was achieved while preserving the same wirelength QoR and at the expense of only a 1% runtime increase.

## VI. RESULTS AND DISCUSSION

### A. Choosing the Agent and Its Parameters

To choose the appropriate agent, its parameters, and the optimal value for the different move types parameters, many experiments were performed; some were already detailed in Sections V-C and V-D. Table III shows the values we use for each of the agent and move parameters.

### B. Comparison to VPR 8

To evaluate *RLPlace*, we compared it to the VPR 8 on both benchmark suites listed earlier in Table II. Fig. 9(a) shows QoR tradeoffs for postplacement estimated wirelength (WL) and CPD versus runtime achieved by *RLPlace* and by VPR 8 on the large VTR benchmarks. It is clear that our runtime/quality

TABLE III  
BEST PERFORMING PARAMETER VALUES OF THE AGENT  
AND THE DIRECTED MOVES

| Parameter             | context         | Value                  |
|-----------------------|-----------------|------------------------|
| high_fanout_limit     | subsection IV-B | 10                     |
| max_range_limit       | subsection IV-B | 3                      |
| high_crit_threshold   | subsection IV-F | 0.7                    |
| agent algorithm       | Eq. 7           | Softmax                |
| agent reward function | Eq. 11          | WLbiased-runtime-aware |
| $\beta$               | Eq. 7           | 100,000,000.           |
| $\alpha$              | Eq. 7           | 3                      |
| $\gamma$              | Eq. 4           | 0.05                   |
| $R_s$                 | Eq. 11          | 0.4                    |

tradeoff curve is below the VPR 8 curve in all tradeoff points. These results can be interpreted in two ways. First, *RLPlace* can achieve the same wirelength and CPD results as VPR 8 in less runtime. Second, it can achieve better QoR than VPR 8 in the same runtime budget. We have confirmed similar results when using *RLPlace* as detailed placer at quench after an analytical placer: HeAP [10].

We also tested *RLPlace* on a different FPGA architecture (Stratix-IV like architecture) with the Titan23 benchmark suite [4]. The results of this experiment are shown in Fig. 9(b). It again shows that *RLPlace* outperforms VPR 8 on both wirelength and CPD for all tradeoff

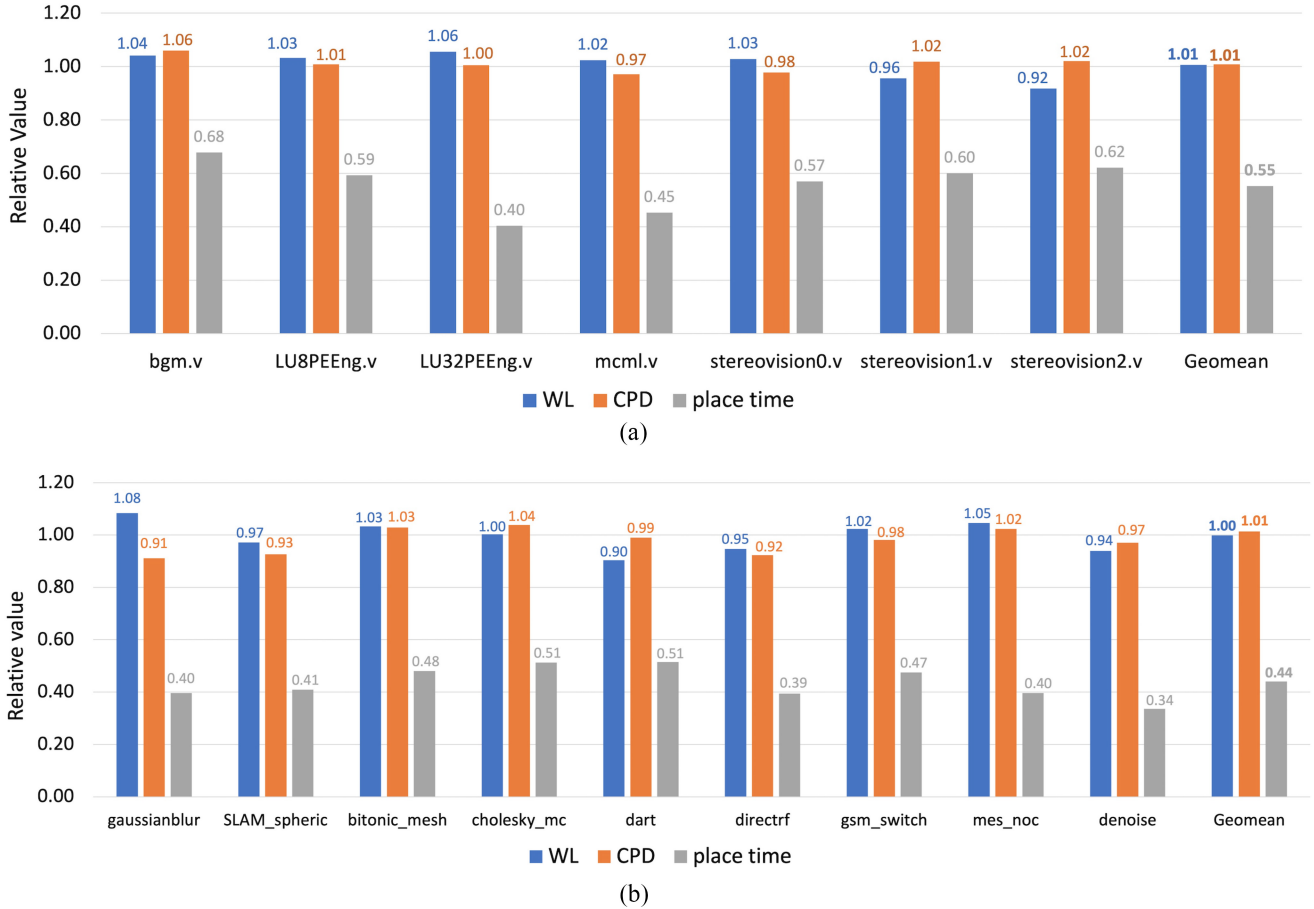


Fig. 10. *RLPlace* per-circuit results normalized to VPR 8 results with default runtime effort for both tools. (a) VTR Benchmark suite. (b) Titan23 Benchmark suite.

points. The gains on the Titan23 benchmarks are greater than those on the smaller VTR benchmarks, indicating that *RLPlace*'s gains tend to increase with design size. To confirm the architecture adaptability of *RLPlace*, we also implemented the VTR benchmarks in a lattice ECP-like FPGA and *RLPlace* achieved similar gains over VPR8 with no hyperparameter retuning. We also measured the runtime overhead of the RL agent to propose a move, calculate its reward, and update the  $Q$  table; on average, the RL overhead is 0.53% for the Titan23 benchmarks.

To quantify the sources of these QoR gains, we performed an experiment using a random agent that chooses move types randomly with equal probabilities (the green line on Fig. 9). The graphs clearly show that about half of the wirelength gains are achieved by using the directed moves themselves (with only a random agent). However, they can be dangerous for timing optimization as they can cause an increase to the CPD compared to VPR 8, particularly for the Titan23 designs. Adding the softmax RL agent in *RLPlace* increased the wirelength gains further and reduced CPD on both the VTR and especially the Titan23 benchmark suites, indicating it improves not only quality but also the scalability and robustness of the algorithm.

Fig. 10 shows the exact per-circuit gains achieved by *RLPlace* normalized to VPR 8 running with its default options. For *RLPlace*, we use a different number of moves per temp than VPR 8 as the new moves/algorithm are able to explore the solution space more efficiently. For the VTR benchmarks [Fig. 10(a)], *RLPlace* was able to achieve same wirelength and CPD results in less runtime for all designs with a geometric average of  $0.55\times$  the VPR 8 placement runtime.

For the Titan23 benchmarks [Fig. 10(b)], we show nine representative designs and the geometric average over all the 23 benchmarks.<sup>2</sup> *RLPlace* achieves the same wirelength and CPD with  $0.44\times$  of the placement runtime compared to VPR 8. Once again, the *RLPlace* gains are greater on the Titan23 designs than on the smaller VTR designs, indicating *RLPlace* is more scalable than VPR 8's placement algorithm.

### C. Agent Behavior

To see how the agent behaves in more detail, Fig. 11 shows how many times each move type was selected by the agent at each annealing temperature for four representative designs.

Fig. 11(a) shows this move type distribution for the stereovision1 design from the VTR benchmark suite.

<sup>2</sup>Full per circuit results can be found at: <https://bit.ly/3xaNYIT>.

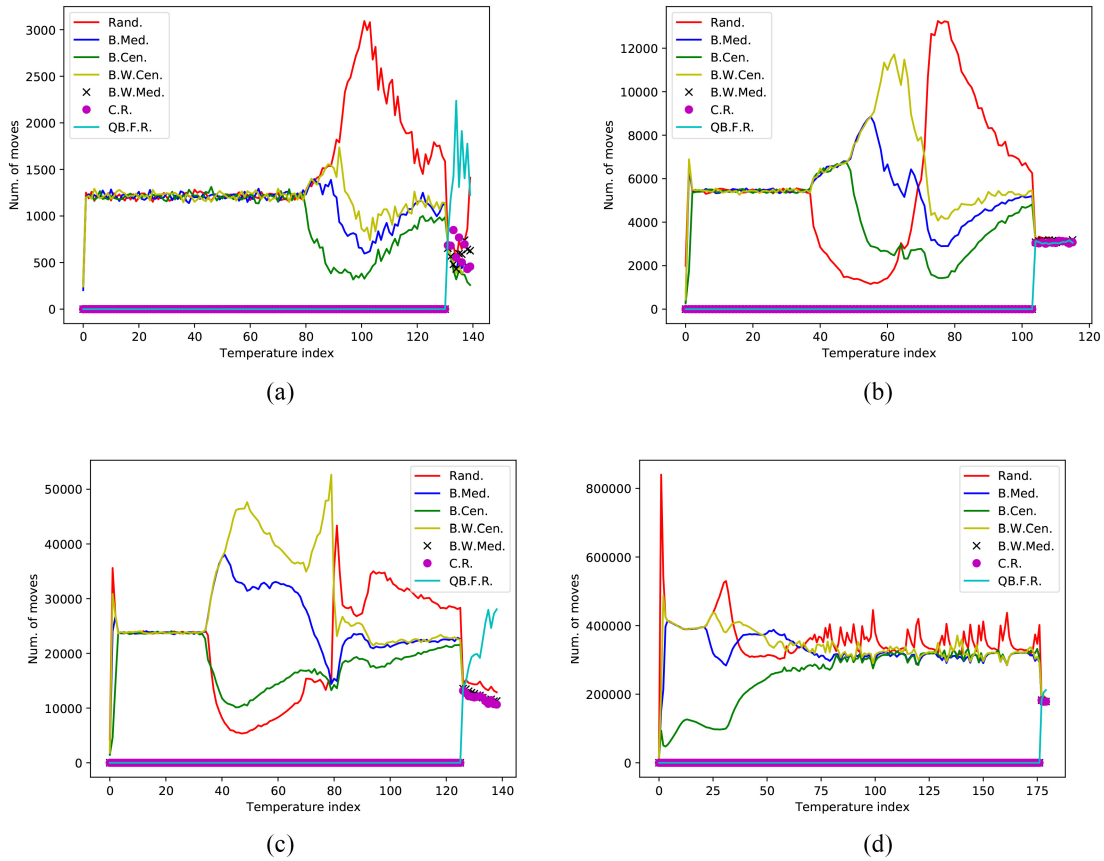


Fig. 11. Number of every move type chosen at each annealing temperature for different designs. (a) stereovision1.v. (b) bgm.v. (c) bitonic\_mesh. (d) directrf\_stratixiv.

The agent starts with equal probabilities for all move types. At mid temperatures, the random, bounded median, and bounded weighted centroid moves are preferred. Later, the random move within a local range dominates. Finally, near the end of the anneal, moves that target more timing optimization (B.W.Med, C.R., and QB.F.R.) are activated when the agent's state is changed and the quasibounded FR move dominates all moves. The agent's behavior is different for the bgm.v circuit (another design from VTR benchmarks) as shown in Fig. 11(b) where random move is not used much early on the anneal but dominates later. Moreover, the agent preferred to choose all moves with equal probability late in the anneal when the timing optimization moves were activated. Although both of these designs target same FPGA architecture (k6\_frac\_N10\_frac\_chain\_mem\_32K\_40nm.xml), the agent decisions were different per circuit and per annealing stage.

Fig. 11(c) and (d) shows same results for two designs from the Titan23 benchmark suite targeting a Stratix-IV like architecture. They show significantly different move type preference throughout the anneal, and the move distribution is different from each other and from the two VTR benchmark designs.

Although the RL agent is not pretrained and starts all cases with zero experience and only learns from the interaction with the environment of a specific placement run, it is able to

adapt dynamically with the optimization process optimizing the expected reward of each action and generates a different set of move decisions for each design. These customized decisions more efficiently explore the solution space, as they lead to higher QoR in less runtime.

## VII. CONCLUSION

We presented *RLPlace*<sup>3</sup>, a novel FPGA placer that utilizes multiple directed move types with an RL agent to control which moves to propose throughout the anneal. Moreover, we presented some techniques to avoid and damp the QoR oscillations that can occur with directed moves by using the bounded range limit technique for most of the moves and by leveraging placement checkpointing. By combining these techniques, *RLPlace* explores the solution space more efficiently than traditional SA-based placers, yielding better QoR at all the CPU runtime points evaluated. *RLPlace* can achieve the same or better wirelength and timing results than VPR 8 at default settings with 45% less runtime for the VTR benchmarks and in 56% less runtime for the (larger) Titan23 benchmarks. It also achieves better QoR than VPR 8 for both timing and wirelength

<sup>3</sup>RLPlace is integrated in the open-source VTR project at <https://github.com/verilog-to-routing/vtr-verilog-to-routing> and is used as the default placer in the master branch (i.e., the latest development code).

if it is given the same runtime as VPR 8. These results are robust for the different FPGA architectures tested and scale well with design size; making it a promising technique to deal with the CAD challenge of ever-growing FPGA capacity.

In our future work, we plan to broaden RLPlace's optimization goals and incorporate additional RL approaches. RLPlace can be made congestion aware by adding an expected routing congestion term to the RL agent's reward function and creating move types that focus on wiring reduction in congested regions. Currently RLPlace moves entire logic, DSP, RAM, and IO blocks. Allowing RLPlace to move individual primitives (such as LUTs, FFs, multipliers, etc.) exposes additional optimization opportunities, and the larger space of perturbations possible will further increase the importance of intelligent move selection. Our current RL agent is table based and learns completely online, starting from scratch on each new design. We plan to investigate alternative agent architectures and combining offline pretraining with online fine-tuning.

## REFERENCES

- [1] A. Boutros and V. Betz, "FPGA architecture: Principles and progression," *IEEE Circuits Syst. Mag.*, vol. 21, no. 2, pp. 4–29, May 2021.
- [2] Xilinx. *Virtex UltraScale+ VUI9P FPGA Product Brief*. [Online]. Available: <https://www.xilinx.com/content/dam/xilinx/publications/product-briefs/virtex-ultrascale-plus-vu19p-product-brief.pdf>
- [3] Intel. *Intel® Stratix® 10 GX 10M FPGA Product Specifications*. [Online]. Available: <https://www.intel.com/content/www/us/en/products/sku/210290/intel-stratix-10-gx-10m-fpga/specifications.html>
- [4] K. E. Murray, S. Whitty, S. Liu, J. Luu, and V. Betz, "TITAN: Enabling large and complex benchmarks in academic CAD," in *Proc. IEEE FPGA*, 2013, pp. 1–8.
- [5] A. Khatkhate *et al.*, "Recursive bisection based mixed block placement," in *Proc. Int. Symp. Phys. Design*, 2004, pp. 84–89.
- [6] Z. Abuwaimer *et al.*, "GPlace3.0: Routability-driven analytic placer for ultrascale FPGA architectures," *ACM Trans. Design Autom. Electron. Syst.*, vol. 23, no. 5, pp. 1–33, 2018.
- [7] W. Li, Y. Lin, and D. Z. Pan, "elfPlace: Electrostatics-based placement for large-scale heterogeneous FPGAs," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2019, pp. 1–8.
- [8] D. Vercruyck, E. Vansteenkiste, and D. Stroobandt, "LIQUID: High quality scalable placement for large heterogeneous FPGAs," in *Proc. IEEE ICFPT*, 2017, pp. 17–24.
- [9] T. Martin, D. Maarouf, Z. Abuwaimer, A. Alhyari, G. Gréwal, and S. Areibi, "A flat timing-driven placement flow for modern FPGAs," in *Proc. 56th Annu. Design Autom. Conf.*, 2019, p. 4.
- [10] M. Gort and J. H. Anderson, "Analytical placement for heterogeneous FPGAs," in *Proc. FPL*, 2012, pp. 1–8.
- [11] C.-W. Pui *et al.*, "RippleFPGA: A routability-driven placement for large-scale heterogeneous FPGAs," in *Proc. ICCAD*, 2016, pp. 1–8.
- [12] Intel *Quartus Prime Pro Edition User Guide: Design Compilation*. [Online]. Available: <https://www.intel.com/content/www/us/en/programmable/documentation/zpr1513988353912.html>
- [13] K. E. Murray *et al.*, "VTR 8: High performance CAD and customizable FPGA architecture modelling," *ACM Trans. Reconfig. Technol. Syst.*, vol. 13, no. 2, pp. 1–55, 2020.
- [14] K. E. Murray, M. A. Elgammal, V. Betz, T. Ansell, K. Rothman, and A. Comodi, "SymbiFlow and VPR: An open-source design flow for commercial and novel FPGAs," *IEEE Micro*, vol. 40, no. 4, pp. 49–57, Jun. 2020.
- [15] M. A. Elgammal *et al.*, "Learn to place: FPGA placement using reinforcement learning and directed moves," in *Proc. IEEE FPT*, 2020, pp. 85–93.
- [16] A. Ludwin, V. Betz, and K. Padalia, "High-quality, deterministic parallel placement for FPGAs on commodity hardware," in *Proc. FPGA*, 2008, pp. 14–23.
- [17] M. An, J. G. Steffan, and V. Betz, "Speeding up FPGA placement: Parallel algorithms and methods," in *Proc. IEEE FCCM*, 2014, pp. 178–185.
- [18] C. Fobel, G. Grewal, and D. Stacey, "A scalable, serially-equivalent, high-quality parallel placement methodology suitable for modern multicore and GPU architectures," in *Proc. 24th Int. Conf. Field Program. Logic Appl. (FPL)*, 2014, pp. 1–8.
- [19] K. Vorwerk, A. Kennings, and J. W. Greene, "Improving simulated annealing-based FPGA placement with directed moves," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 28, no. 2, pp. 179–192, Feb. 2009.
- [20] A. Ludwin and V. Betz, "Efficient and deterministic parallel placement for FPGAs," *ACM Trans. Design Autom. Electron. Syst.*, vol. 16, no. 3, pp. 1–23, 2011.
- [21] J. Yuan, J. Chen, L. Wang, X. Zhou, Y. Xia, and J. Hu, "ARBSA: Adaptive range-based simulated annealing for FPGA placement," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 38, no. 12, pp. 2330–2342, Dec. 2019.
- [22] A. Alhyari, A. Shamli, Z. Abuwaimer, S. Areibi, and G. Gréwal, "A deep learning framework to predict routability for FPGA circuit placement," in *Proc. IEEE FPL*, 2019, pp. 334–341.
- [23] A. Y. Al-Hyari, A. Shamli, T. Martin, S. Areibi, and G. Gréwal, "An adaptive analytic FPGA placement framework based on deep-learning," in *Proc. MLCAD*, 2020, pp. 3–8.
- [24] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in *Proc. 19th Int. Conf. World Wide Web*, 2010, pp. 661–670.
- [25] D. Silver *et al.*, "A general reinforcement learning algorithm that masters Chess, Shogi, and go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.
- [26] Z. He *et al.*, "Learn to floorplan through acquisition of effective local search heuristics," in *Proc. IEEE ICCD*, 2020, pp. 324–331.
- [27] A. Mirhoseini *et al.*, "Chip placement with deep reinforcement learning," 2020. [Online]. Available: [arXiv:2004.10746](https://arxiv.org/abs/2004.10746).
- [28] H. Liao, W. Zhang, X. Dong, B. Póczos, K. Shimada, and L. B. Kara, "A deep reinforcement learning approach for global routing," *J. Mech. Design*, vol. 142, no. 6, 2020, Art. no. 061701.
- [29] H. Wang *et al.*, "GCN-RL circuit designer: Transferable transistor sizing with graph neural networks and reinforcement learning," in *Proc. IEEE DAC*, 2020, pp. 1–6.
- [30] Z. He, L. Zhang, P. Liao, Y. Ma, and B. Yu, "Reinforcement learning driven physical synthesis," in *Proc. IEEE ICSICT*, 2020, pp. 1–4.
- [31] K. E. Murray and V. Betz, "Adaptive FPGA placement optimization via reinforcement learning," in *Proc. MLCAD*, 2019, pp. 1–6.
- [32] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
- [33] S. Kirkpatrick, D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [34] K. E. Murray and V. Betz, "TATUM: Parallel timing analysis for faster design cycles and improved optimization," in *Proc. IEEE FPT*, 2018, pp. 110–117.
- [35] V. Betz, J. Rose, and A. Marquardt, *Architecture and CAD for Deep-Submicron FPGAs*, vol. 497. New York, NY, USA: Springer, 2012.
- [36] M. Wainberg and V. Betz, "Robust optimization of multiple timing constraints," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 34, no. 12, pp. 1942–1953, Dec. 2015.
- [37] R. F. Hentschke and R. Reis, "Improving simulated annealing placement by applying random and greedy mixed perturbations [IC layout]," in *Proc. IEEE SBCCI*, 2003, pp. 267–272.
- [38] G. Chen and J. Cong, "Simultaneous timing-driven placement and duplication," in *Proc. FPGA*, 2005, pp. 51–59.
- [39] A. Marquardt *et al.*, "Timing-driven placement for FPGAs," in *Proc. FPGA*, 2000, pp. 1564–1569.
- [40] K. E. Murray *et al.*, "Timing-driven titan: Enabling large benchmarks and exploring the gap between academic and commercial CAD," *ACM Trans. Reconfig. Technol. Syst.*, vol. 8, no. 2, pp. 1–10, 2015.
- [41] P. R. Montague, "Reinforcement learning: An introduction, by Sutton, RS and Barto, AG," *Trends Cogn. Sci.*, vol. 3, no. 9, p. 360, 1999.